PRODUCT CONCEPT / HUMAN READING COPY

# CodeSentinel

## AI-assisted regression detection, diagnosis, and verified repair

**The idea in one minute**

CodeSentinel is a developer tool that watches a software repository, identifies code changes, runs relevant tests, and uses an NVIDIA open model served through Nebius Token Factory to help explain failures and propose repairs. Its central promise is to show developers what a change broke, provide evidence for why it broke, and verify whether a proposed fix restores expected behavior.

**Document status:** Concept and planning document. It is not evidence that any feature has already been implemented.
**Working name:** CodeSentinel. Check name, domain, and trademark availability before making the name permanent.
**Proposed hackathon track:** Coding and Agentic Engineering, subject to rechecking the official requirements.

## 1. The problem

A change can compile and still break a different page, API, or user journey. Developers may test only the area they edited and miss an indirect regression. Once a problem appears, they often have to connect the code diff with test failures, logs, and the affected workflow by hand.

CodeSentinel aims to reduce this investigation effort by linking those pieces together. It cannot guarantee that every future bug will be predicted. It can detect observed failures, compare behavior with a baseline, prioritize likely risk, and explain evidence collected by deterministic tools.

## 2. Who it is for

- Individual developers building web applications.
- Small teams shipping frequent code changes.
- Hackathon teams using AI coding assistants who want to catch regressions quickly.
- Maintainers who need an incident history tied to repository changes.

The first version should focus on one repository and one supported application/test setup instead of claiming universal language or framework support.

## 3. How a user experiences it

1. A developer connects a GitHub repository and configures commands and critical user flows to test.
2. A push or pull request generates a repository event. The backend verifies the webhook signature and records the event safely, including duplicate-delivery handling.
3. CodeSentinel identifies the commit or pull-request diff and queues a test run in an isolated environment.
4. The runner executes configured checks, such as build, lint, type-check, unit tests, and a small number of Playwright browser journeys.
5. If a check fails, CodeSentinel captures evidence: command, exit code, relevant output, failed test, route if applicable, and run/commit identifiers.
6. The NVIDIA model served through Nebius Token Factory receives a bounded, redacted evidence bundle and proposes a likely cause, affected behavior, confidence, and safe remediation steps.

7. The dashboard distinguishes actual observations from model hypotheses. A proposed patch may be created only in an isolated branch or worktree and under explicit repository permissions.
8. The relevant tests run against the proposed patch. A fix is marked verified only when configured checks pass; otherwise the result remains failed or unverified.

## 4. What makes the idea stronger than a tracker

The differentiating workflow is evidence-led diagnosis and verification. The system should:

- Link code changes to potentially affected routes, APIs, tests, and user journeys.
- Select relevant tests where safe, and explain why those tests were selected.
- Show actual failure evidence before asking the model for an explanation.
- Label confidence and uncertainty instead of presenting model output as fact.
- Verify proposed patches by running tests again.
- Keep a history of confirmed regressions and verified fixes for the repository.

These methods are not all individually novel. The project must demonstrate that the combined workflow is useful, reliable, and coherent.

## 5. First-version scope

### Included in the MVP

- Connect one GitHub repository through a narrowly scoped integration.
- Accept push and pull-request webhooks with signature validation.
- Store project configuration, events, test runs, findings, and AI analyses.
- Run configured checks in an isolated runner with resource and time limits.
- Support a small, explicit set of browser workflows through Playwright.
- Call at least one NVIDIA open-source model through Nebius Token Factory for evidence-grounded diagnosis.
- Show run status, failed checks, evidence, AI explanation, and remediation proposal in a dashboard.
- Re-run selected checks against a candidate fix in isolation.

### Defer until the MVP is reliable

- Local monitoring of uncommitted files and IDE plugins.
- Multi-repository organization features and broad language support.
- Deeper code graphs and production telemetry integrations.
- Additional notification channels, automatic merging, or production deployment.

## 6. What it does not promise

- It does not prove the absence of all bugs or predict every future failure.
- It does not replace a team's test suite or production observability platform.
- It does not silently edit or deploy production code.
- It does not execute repository code with unrestricted host, secret, or network access.
- It does not claim that a fix works merely because the model says it does.

## 7. Role of Nebius and NVIDIA

The model call must be part of the actual product workflow, not a decorative demo. CodeSentinel uses an NVIDIA open-source model served through Nebius Token Factory to interpret diffs and test evidence, structure a diagnosis, and propose remediation. The exact model identifier, endpoint, parameters, limitations, and observed behavior must be confirmed and recorded during implementation.

Do not hard-code a model name from an example without confirming that it is currently available to the team's account. Likewise, do not claim sandbox execution or another platform capability unless the application actually uses it.

## 8. How to demonstrate the value

Use a small, controlled sample web application with a known-good baseline and three user journeys. Introduce one deliberate regression that breaks a journey outside the edited component. Demonstrate the following sequence:

- The repository event is captured.
- The relevant test fails reproducibly.
- The AI explanation cites real evidence and identifies a plausible cause.
- A candidate fix is proposed in isolation.
- Tests pass after the patch, or the system honestly reports that the attempt did not work.

Track factual demo metrics such as detection latency, number of tests run, test outcome, model latency, and token usage. Do not invent time-saved or accuracy claims from a single demo.

## 9. Success criteria

The first version is successful when a reviewer can reproduce the end-to-end workflow from a clean setup; verify the real Nebius/NVIDIA runtime integration; distinguish actual failures from model-generated hypotheses; see that unsafe production actions are avoided; and understand the result without needing to study the internal architecture.

## 10. Team decisions still required

The companion technical documents provide recommended defaults, but they do not pretend that the team has already selected every tool or policy. Before implementation, confirm these decisions and record them in the architecture and README:

| Decision | What the team must confirm |
|---|---|
| Application and stack | Repository/framework, frontend and backend stack, hosting target, and database. |
| Testing setup | Supported build/test commands, critical browser journeys, and the limits of the test runner. |
| GitHub integration | Webhook or app integration, required permissions, retry behavior, and idempotency strategy. |
| AI integration | NVIDIA open model available to the team, Nebius endpoint, configuration, and usage limits. |
| Execution and safety | Isolation model, secret handling, network access, time/resource limits, and artifact retention. |
| Repair permissions | Whether patches are suggestion-only or may be committed to a separate branch after approval. |
| Notifications and demo | Notification channel, sample application, reproducible failure, and demo metrics. |

# 11. Reference links and quick reference

These links are starting points for checking platform capabilities and implementation details. Re-check current documentation during development; the project overview is a planning document, not a guarantee that a feature is available in every account or environment.

- Hackathon page
- Official rules
- Hackathon tips
- Nebius Token Factory / NVIDIA Nemotron overview
- Token Factory documentation
- GitHub webhook signature validation
- Playwright documentation

## How to interpret a result

| Result type | Meaning |
|---|---|
| Observed failure | A deterministic check actually failed. The report should include the command or test, exit status, and relevant evidence. |
| AI diagnosis | A model-generated explanation of the evidence. It is a hypothesis until supported by reproduction or further checks. |
| Proposed fix | A candidate code change. It has not succeeded merely because the model generated it. |
| Verified fix | The agreed set of relevant checks passed against the candidate change. This is evidence of repair, not a guarantee that no other bug exists. |

## The first build milestone

| Step | Done when |
|---|---|
| 1. Connect repository | A push or pull request is received and recorded safely. |
| 2. Reproduce a regression | A configured test fails against a deliberately broken change and passes on the known-good baseline. |
| 3. Explain with evidence | The NVIDIA model receives the real diff and test evidence and returns a useful, clearly labeled diagnosis. |
| 4. Verify a repair | A proposed patch is tested in isolation and the dashboard reports the actual result. |

### Guiding principle

Report what the system actually observed, separate that evidence from AI-generated hypotheses, and call a fix verified only after the relevant checks pass.